

Programming Assignment 2: Distributed Bellman-Ford

Due: July 14, 2026.

1 Introduction

In this assignment you will implement a simple version of the distributed Bellman-Ford algorithm. The algorithm will operate as a set of user-level processes for jobs, and another process which simulates the network. More specifically, there is a process associated with each node, which gets, as input, the cost of links connecting it to its neighbors. The task is, for each node, to compute a root, its next link in the shortest path connecting it to its root, and the length of that shortest path.

The node processes communicate by exchanging 127-byte messages through a central process, called *netproc*, that emulates the network. The network emulator *netproc* is provided (see the resources section of the assignment in the moodle site); your job is to implement the node processes.

An execution of the system has the following general structure. First, *netproc* is invoked, opens a TCP socket bound to port 6789. Then the node processes are started. Each node process initially `connect()` to the *netproc* socket and sends its ID (see below) by writing it (as an `int` converted by `hton()`) to the socket; thereafter, to send a message to a neighbor, the node writes the message on the TCP channel its message, *preceded by a 1-byte prefix* whose contents is the link number to which the message is sent. For example, the following piece of code sends a message (assumed to be already filled) `msg` to the 7th neighbor, via the socket `s`, connected to *netproc*:

```
char msg[127];
char dest;
// ...
dest = 7;
write(s, &dest, 1)
write(s, msg, 127);
```

(Using `write()` this way is bad practice: We recommend using Stevens' `written()`, see [2].)

The *netproc* process will deliver the message to the receiving node via the TCP channel. In the incoming message, the leading byte will be the link number at the receiver (which, in general, is different than the link number at the sender).

Notes.

- To send a message to all neighbors, use `0xFF` as the first byte (link number).
- The socket option `TCP_NODELAY` should be set. See the FAQ section in the moodle site.

Similarly to BPDU in the Spanning-Tree Protocol of IEEE 802.1, update messages sent by a node contain the following information:

1. `myRoot` - Smallest ID known to the process.
2. `myCost` - cost of best known path to `myRoot`.
3. `myID` - Identification number of the sending process.
4. `expTime` - How long does the message have until expiration.

A message is sent if and only if one of the following holds:

- The process view (namely, the value of `myRoot` or `myCost`) has changed.
- Every `HELLO_TIMEOUT` (a predefined constant) period of time.

The hello timer is used to make sure that a node sends out a message at least once every `HELLO_TIMEOUT` seconds. Thus the timer should be reset to `HELLO_TIMEOUT` whenever a message is sent by the node to its neighbors.

To allow recovery in case the root crashes, processes must also keep track of the expiration time as implied by `expTime` in the message. Whenever a message is sent, it is sent with the `expTime` value adjusted to account for the time since it was received (for example, if a message is received at time 3 with `expTime = 100`, then a message sent at time 12 will have `expTime = 100 - (12 - 3) = 91`). The expiration time may increase only when a fresher message (i.e., with a larger `expTime` value) arrives for with the same ID of `myRoot`. Whenever `expTime` expires (reaches 0), the process sets `myRoot` to its own ID, and `myCost` to 0. A message whose `myRoot` is its own ID has expiration time infinity, and the process that sends such a message sets `expTime=ROOT_TIMEOUT`, which is another predefined constant (usually 3 times `HELLO_TIMEOUT`). The constants are defined in the `bf.h` file, which can be found in the resources section of the assignment in the moodle site.

2 User Interface

The network topology is specified by a single configuration file, used both by the `netproc` process and the node processes. The configuration file is just a list of links: each row is a comma-separated triple: endpoint 1, endpoint 2, and cost.

You will write code for a process that receives command-line arguments as follows:

```
nodeproc netproc_address node_id lifetime cost1 [cost2...]
```

Where:

- *nodeproc* is the file name of your executable for the node process.
- *netproc_address* is the IP address of the host that runs `netproc` (if the same machine, use 127.0.0.1). Remember that the port number on which the `netproc` process listens is 6789.
- *node_id* is the ID you assign to the current process (an integer).
- *lifetime* is the time, in seconds, the process should be up before it gracefully terminates (an integer).
- The rest of the line consists the costs of the incident links: *cost1* is the cost of the link at port 1, *cost2* is the cost of the link at port 2 etc. All costs are non-negative integers. These costs can be derived from the configuration file by running the script `costs.sh` which we provide. To use it, you can run `./costs.sh config.csv ID` where `config.csv` is the configuration file and `ID` is the node ID. Alternatively, you can run the script in the command line as follows:

```
nodeproc netproc_addr node_id lifetime $(./costs.sh topology.csv node_id)
```

When the process runs, it prints out (to the standard output) a line for every significant event. The event is described as a list of tab-separated fields: the time is the first field always. The other fields depend on the nature of the event: either a message send or an update of the root ID or the distance to the root. In case of an update, the new values are printed. In any case, the time of the event is printed. For example, a typical transcript may look like the following.

```
> nodeproc 132.66.48.12 100 5432 10 7 3 1
time=120.0 Root=100 parent=NULL distance=0
time=120.1 Message sent to all neighbors
time=123.1 Message sent to all neighbors
time=125.3 Root=95 parent=1 distance=3
time=125.4 Message sent to all neighbors
time=125.6 Root=95 parent=217 distance=2
```

```
time=130.0 Lifetime expired. Shutting down.  
>
```

Please follow the formatting guidelines below to print:

```
printf("time=%.01f\tRoot=%d\tparent=%d\tdistance=%d\n",  
       time_ms/1e3f, root_id, parent_id,distance)  
printf("time=%.01f\tMessage sent to all neighbors\n", time_ms / 1e3f,)  
printf("time=%.01f\tLifetime expired. Shutting down.\n", time_ms / 1e3f,)
```

Obviously, for debugging purposes more printouts may be helpful.

Notes:

1. Time is obtained using `clock_gettime()` with `CLOCK_REALTIME`. Take a look at the man page for explanation and examples. This can be used to calculate timer's status. See also the FAQ section in the moodle site for some tips and pointers.
2. The initial state is printed out. Also note that special values are printed ("NULL").
3. In the third line, an update is sent due to a HELLO timer expiring. However, in lines 2 and 5, updates are sent to all neighbors due to topology change.

Tips.

- It may be convenient to work with multiple threads. If you choose a single-threaded process, you need to design a timer mechanism that allows for multiple timeouts: in that case you will be using only one timer (implicit in the `select()` system call) when you block, which means that you have to manage a queue of timeout values that should occur in the future (e.g., the HELLO, ROOT_TIMEOUT and lifetime timeouts).
- As the application uses TCP, it is important to correctly identify message boundaries. You can opt for either fixed-length messages, or a length field in the header, or a special byte sequence to mark the end of a message.
- Test your program with non-trivial topologies. Try to run the program with different initial distance values and with loops. We shall provide the netproc process and a few configuration examples.
- Note that node processes (and netproc) may run on multiple machines.
- A process should be terminated when its lifetime expires. Make sure to properly close all sockets and release other used resources.

The exercise will be graded first and foremost by implementing functionality, which will be tested under simple and extreme scenarios. Efficiency, both of communication and coding, is also part of the grade. Good coding practices are required as well. Points will be taken off for lack of documentation.

3 What to use and what to submit

In the moodle site there is the Resources section, with all files required for the project, as well as Frequently Asked Questions with implementation tips.

Submission will be done in a single zip archive on moodle. The zip archive should contain the following files, *and them only*:

- README: contains the full documentation, including names and ID numbers of the team members. We will read this file.
- Makefile: used to build your application under the name `bfproc`.
- All project `*.h` and `*.c` (or `*.cpp`) source code files.

The documentation should describe whatever is unusual about your implementation (such as additional or missing features). Include brief explanation of your algorithm and data structures in the code. Describe your packets structure and the way you implemented multiple timeouts.

References

- [1] *Beej's Guide to Network Programming*, by Brian “Beej Jorgensen” Hall. Section 7.3 explains `select()`. Available at <https://beej.us/guide/bgnet/>.
- [2] *Unix Network Programming vol. 1*, by W. R. Stevens. It's written for BSD Unix, but it's still one of the best sources on socket programming.
Available from github: <https://github.com/ben-elbert/books/blob/master/>.